week05

상속(extends)

어떤 클래스의 기능을 다른 클래스가 사용할 수 있게 하는 기법

```
clss BoyGroups extends Idol{
    //상속받은 생성자
    BoyGroup(
        String name, //데이터를 받는 입구
        int membetsCount,
        ) : super( //부모에게 배달
    name,
    membersCount,
    );
void sayMale(){ //상속받지 않은 기능
print('저는 남자 아이돌입니다.');
    }
}
```

오버라이드

부모 클래스 또는 인터페이스에 정의된 메서드를 재정의할 때 사용한다

override 키워드 생략 가능하다

```
class GirlGroup extends Idol {
  GirlGroup(
    super.name,
    super.membersCount,
  ); // 위의 코드보다 더 간결하게 작성

  // 1 override 키워드를 사용해 오버라이드합니다.
  @override
  void sayName() {
    print('저는 여자 아이돌 ${this.name}입니다.');
```

```
  }
}
```

인터페이스

상속은 공유되는 기능을 이어받는 개념이지만 인터페이스는 공통으로 필요한 기능을 정의만 해두는 역할을 한다

상속은 단 하나의 클래스만 할 수 있지만 인터페이스는 적용 개수에 제한이 없다

```
    class GirlGroup implements Idol{
        final String name;
        final int membersCount;

    GirlGroup(
        this.name,
        this.membersCount,
        );

    void sayName(){
        print('저는 여자 아이돌 ${this.name}입니다.');
        }

    void sayMembersCount(){
        print('${this.name} 멤버는 ${this.membersCount}명입니다.');
        }
    }
```

- 상속은 부모 클래스의 모든 기능을 재정의할 필요 없지만 인터페이스는 모든 기능을 정의해야 한다.
- 애초에 반드시 재정의할 필요가 있는 기능을 정의하는 용도가 인터페이스이기 때문이다.

믹스인

특정 클래스에 원하는 기능들만 골라 넣을 수 있는 기능이다

추상

상속이나 인터페이스로 사용하는 데 필요한 속성만 정의하고 인스턴스화할 수 없도록 하는 기능

메서드 정의를 자식 클래스에 위임할 수 있다